BLUVFI

How Bluvfi Works

A technical overview of the product, its architecture, and the payment systems underneath it — written for someone who wants the real mechanics, not the marketing version.

September 2, 2026

1. What Is Bluvfi

Bluvfi is an AI-driven financial assistant. It combines a chat interface — where a user can ask an AI agent to pay a bill, check a balance, or buy a gift card — with a conventional dashboard that offers the same actions directly. Both surfaces are backed by the exact same underlying logic; the chat interface isn't a thin wrapper that calls a different, simpler code path than the dashboard does.

Core capabilities

- **Digital products (Bills)** — gift cards, mobile top-ups, and eSIM data plans via Bitrefill — 10,000+ products, paid in USDC/USDT across several chains, native crypto, or XRP.
- **AI assistant** — a chat agent with dozens of callable tools spanning bills, investments, banking, and DeFi trading.
- **Investments / portfolio** — stock and ETF-style positions, DeFi vault positions, and balance tracking across wallets.
- **Banking** — cross-border money movement — currently shown as "Coming Soon" in the UI while the backing providers are finished and verified.
- **History** — a unified transaction ledger across every payment type the app supports.

> **Design principle worth knowing up front**
> Wherever the dashboard and the AI both need to do the same thing — start an XRP purchase, detect a failed payment, recover stuck funds — that logic lives in exactly one shared function, called identically by both. This shows up repeatedly through the rest of this document, and it's deliberate: two independent implementations of the same financial logic drift apart over time, and drift is exactly where bugs involving real money hide.

2. Architecture at a Glance

Application layer

- **Framework** — Next.js (App Router), TypeScript throughout, deployed on AWS Amplify.
- **Frontend** — React with Tailwind CSS; a dashboard (Bills, Investments, Banking, History) plus a persistent chat panel.
- **Database** — Postgres, hosted on Neon, accessed through Drizzle ORM with a versioned migration history.
- **AI layer** — the Vercel AI SDK's tool-calling framework, with automatic provider fallback (Google, OpenAI, Qwen) so a single provider outage doesn't take chat down.

Authentication and wallets

Privy provides both authentication and embedded, non-custodial wallets — every user gets an EVM wallet and a Solana wallet that Privy generates and holds the keys for on the user's behalf, in a way that Bluvfi's own backend never touches directly. Server-side requests are authenticated by verifying a Privy-issued JWT, accepted either as an Authorization: Bearer header or via Privy's own session cookie.

External integrations

Integration	Purpose
Bitrefill	Gift cards, mobile top-ups, and eSIMs — reached through Bitrefill's own MCP (Model Context Protocol) tool interface.
bluvfi-xrpl	A separate, Bluvfi-operated service that generates and custodies XRPL wallets, used only for the "Pay with XRP" feature.
Circle Gateway	USDC settlement rails used by several payment tools.
Credible Finance / Ripple ODL	Banking providers — integrated in code, not yet exposed in the live UI.
Velvet, dYdX, xStocks, and others	DeFi trading, perpetuals, and tokenized-equity integrations, each with their own dedicated AI tools.

3. Digital Products — Bills

The Bills feature is a storefront for Bitrefill's catalog: gift cards, mobile airtime top-ups, and eSIM data plans, filterable by country and category. A purchase can be started from the dashboard's Browse tab or by asking the AI ("buy me a $20 Steam gift card"). Either way, it resolves to the same purchase logic and lands in the same order history.

Three tiers of payment method

1. Automatic-pay USDC/USDT (Base, Ethereum, Arbitrum, Polygon, Solana, and others) — paid automatically through the user's own Privy wallet. If the wallet needs native gas to cover the transaction and doesn't have it, the user is asked to cover that too. If that still isn't possible, the payment falls back to an ArcKit-managed EOA as a last resort. This three-layer cascade (Privy wallet -> user-paid gas -> ArcKit EOA fallback) exists so a payment doesn't simply fail the first time a wallet is short on gas.
2. Address-based methods (Bitcoin, Lightning, TON, Litecoin, Dogecoin, BSC USDT/USDC, and others) — Bitrefill returns a deposit address and the user sends funds manually, the same pattern used by most crypto payment processors.
3. Pay with XRP — a Bluvfi-specific option, not a payment method Bitrefill itself understands. Covered in full in the next section, since it's the most involved payment path in the app.

4. Pay With XRP

From the user's point of view, paying with XRP looks identical to any other address-based method: a deposit address appears, a short countdown starts, and once the payment is detected the order completes. What actually happens underneath is more involved, and is deliberately never described to the user in those terms — the words "swap," "bridge," and "NEAR Intents" never appear in any user-facing text, in either the dashboard or the AI's chat responses.

What happens, step by step

1. A Bitrefill invoice is created for the product, priced in USDC and settling on one specific chain (Solana, Arbitrum, Polygon, Ethereum, BSC, or Base). Several chains are tried automatically, most-reliable-first, until one returns a working invoice and a working price quote — this happens before any wallet is ever shown to the user, since a Bitrefill invoice is bound to one settlement chain the moment it's created and can't be redirected mid-flight.
2. bluvfi-xrpl generates a brand-new, one-time XRPL deposit wallet for this purchase and reports the exact amount of XRP required — the XRP Ledger's ~1 XRP account reserve, a small safety buffer, and the amount actually needed for the currency conversion, all added together.
3. The user sends that exact amount of XRP to the deposit address, within roughly 7.5 minutes.
4. Once the deposit is detected, bluvfi-xrpl converts the XRP into USDC (through NEAR Intents, an external liquidity network) and forwards it directly to the Bitrefill invoice's own payment address.
5. Bitrefill detects the USDC arriving and fulfills the order — issuing the gift card code, crediting the airtime, or delivering the eSIM.

> **Minimum purchase amount**
> A purchase below roughly $5 cannot be paid with XRP. This isn't an arbitrary limit — the currency-conversion network enforces its own minimum bridgeable amount (observed live at roughly $2.75), and $5 is set as a deliberate buffer above that observed floor, since the exact number has been seen to drift over time. If the user picks an amount that's too small, this is caught and explained before any wallet is even created — not discovered later.

Where XRP purchases can go wrong, and what happens then

Two distinct things can fail: the underlying settlement chain might not be available (handled silently by trying the next chain in the list, before the user ever sees a wallet), or the XRP-to-USDC conversion can fail after the user has already sent XRP — a rate guard rejecting an unfavorable quote, or the payment window simply closing. In that second case, the user's funds never left bluvfi-xrpl's custody, but the purchase itself has failed and needs to be surfaced and recoverable.

5. Failure Detection & Fund Recovery

A failed-after-funding XRP purchase is the one payment scenario in the app where money genuinely gets "stuck" — safe, but inert — unless the app actively notices and offers a way to move it. Bluvfi has no automatic sweep for this; nothing recovers on its own. So the reliability of this feature comes entirely from making sure the failure is always detected, from wherever the purchase happened, and that recovering it is always reachable.

Four independent detection surfaces, one shared outcome

- **An async webhook** — bluvfi-xrpl notifies Bluvfi's backend the moment a conversion fails, refunds, or expires — this works even if nobody has the app open.
- **The dashboard's live status poll** — while a purchase is actively in progress in the browser, the payment screen polls the wallet's real status directly, independent of Bitrefill's own (much slower) invoice status.
- **The AI's own status check** — when the AI is asked to confirm a payment, it performs the same live check before reporting back to the user.
- **The recovery list itself** — the very act of opening the recovery list in the Bills tab re-checks any payment that's still unresolved — a true catch-all for the case where none of the other three ever ran (for example, the tab was closed the instant the deposit was sent).

All four write to the same order record through one shared function, so however the failure was discovered, the rest of the app agrees about what happened.

Recovering the funds

A dedicated list in the Bills tab shows every payment with money still recoverable, regardless of whether it was started from the dashboard or from a chat conversation with the AI, and regardless of whether the browser session that started it is even still open. Recovering moves the stuck amount to the user's own persistent Bluvfi XRP wallet — the same wallet shown in the sidebar — with one click. The AI can also do this directly in a chat conversation, but only after stating the exact amount to the user and receiving explicit confirmation; it never recovers funds unprompted.

> **What is NOT currently recoverable**
>
> The XRP Ledger's account reserve (roughly 1 XRP) attached to a failed purchase's one-time wallet is not recoverable today, by the user or automatically. Reclaiming a ledger reserve requires deleting the account entirely, and the only mechanism that does this is scoped to successful, completed purchases — a wallet whose conversion failed never qualifies. This is a known, acknowledged limitation rather than an oversight, and would need a change on bluvfi-xrpl's side (the custodial wallet service) to close.

6. The AI Agent

The AI is not a chatbot layered on top of the app — it's given direct, callable tools for the same actions the dashboard exposes: searching products, starting a purchase, checking payment history, managing DeFi positions, and more. Its tool descriptions are written as operating instructions, not just documentation — several explicitly forbid the AI from naming internal mechanisms ("swap," "bridge," "NEAR Intents") to keep its explanations consistent with what the dashboard shows.

How the AI is allowed to move money

This differs meaningfully by wallet type, because of who actually holds the private keys:

- **EVM and Solana wallets** — are non-custodial Privy embedded wallets — Bluvfi's backend, and therefore the AI, never holds these keys. Any transfer the AI prepares from a user's own wallet is built as an unsigned transaction the user must sign themselves; the AI cannot execute one unilaterally, as a matter of cryptography, not policy.
- **XRP wallets** — are the one case where Bluvfi's own infrastructure (bluvfi-xrpl) holds the keys on the user's behalf. This makes AI-initiated XRP recovery technically possible without a user signature — so the safeguard there is a behavioral one, enforced by the tool's instructions requiring explicit confirmation before any transfer, rather than a cryptographic wall. The scope is also hard-limited: XRP can only ever move between two wallets the same user already owns (a stuck purchase wallet back to their own sidebar wallet) — there is no tool, anywhere in the app, that sends XRP to an external address.

Provider fallback

The AI can be backed by more than one model provider. If the configured provider is unavailable, the app switches to a fallback automatically, without interrupting the conversation or affecting per-user rate limits.

7. Banking

Two banking providers exist in the codebase — Credible Finance and Ripple Payments ODL — but the Banking tab in the live app currently shows a "Coming Soon" placeholder rather than either integration. Ripple ODL in particular has not yet been verified against a live environment; several implementation details (exact field names, webhook signing scheme) are marked as unconfirmed pending real API access. Keeping the feature gated in the UI means the backend work can continue without any risk of an unverified banking flow reaching a real user.

8. Data Model (Selected Tables)

Table	Purpose
payments	The unified transaction ledger — every payment type in the app (bills, investments, transfers, banking, XRP purchases) writes here, which is what both the History screen and the AI's payment-history tool read from.
bitrefill_orders	Full lifecycle of a Bitrefill purchase, from creation through fulfillment or failure — includes the XRP-specific linkage fields (which XRPL wallet funded it, whether its stuck funds have been recovered).
xrpl_sidebar_wallets	One persistent XRPL wallet per user, created the first time it's needed — this is the destination for any XRP recovery.

9. A Note on How This Document Should Age

This is a snapshot of the system as built, not a permanent specification. Payment method availability (particularly which chains can settle a Bitrefill invoice, and what a bridging network's minimum amount is) has already been observed to shift over time and is monitored rather than assumed fixed. The clearest signal that something described here has changed is the code itself — anything in this document should be checked against the current implementation before being treated as ground truth for a decision with real financial consequences.